

TESI FINALE DI PROGETTAZIONE DI SISTEMI OPERATIVI

Programmazione Concorrente nel Linguaggio Java

Approfondimento sull'utilizzo di Processi Leggeri in Java

Studente:

Tommaso Cosimi

Matricola 191739

Docente:

Prof.ssa Letizia Leonardi

Codocente:

PhD Silvia Cascianelli

Indice

1	Introduzione	1
1.1	Algoritmo, Programma, Processo	1
1.2	Stati di un Processo	3
1.3	Esecuzione Sequenziale e non Sequenziale	5
1.3.1	Processi Sequenziali	5
1.3.2	Processi non Sequenziali	5
1.4	Processi Interagenti	6
1.4.1	Interazione Indiretta (Competizione)	6
1.4.2	Interazione Diretta (Cooperazione)	7
1.4.3	Interazione Indesiderata (Interferenza)	7
1.5	Sincronizzazione tra Processi	7
1.5.1	Sezione Critica	8
1.5.2	Strumenti di Sincronizzazione	8
1.5.3	Modelli Logici per la Concorrenza	13
1.6	Processi Pesanti e Processi Leggeri	14
1.6.1	Processi Pesanti	14
1.6.2	Processi Leggeri - “Thread”	14
2	Thread in Java	16
2.1	Creazione di un Processo Leggero	16
2.1.1	Estensione della Classe Thread	17
2.1.2	Implementazione dell’Interfaccia Runnable	18
2.1.3	Considerazioni Pratiche ed Esecuzione	19
2.2	Primitive per la Gestione dell’Esecuzione	19
2.2.1	Considerazioni	21
3	Sincronizzazione tra Processi	23
3.1	Il Costrutto <i>Synchronized</i>	23
3.1.1	Metodi <i>Synchronized</i>	23
3.1.2	Blocchi <i>Synchronized</i>	24

3.1.3	Le Primitive <code>wait()</code> , <code>notify()</code> e <code>notifyAll()</code>	24
3.1.4	Rientranza del Costrutto <i>Synchronized</i>	24
3.1.5	Considerazioni	26
3.2	L'Interfaccia <code>Lock</code>	26
3.2.1	Categorie di <i>Lock</i>	27
3.2.2	Equità del <i>Lock</i>	27
3.2.3	Primitive di <code>Lock</code>	28
3.2.4	Variabili Condizione	28
3.3	La Classe <code>Semaphore</code>	29
3.3.1	Equità di un Semaforo	29
3.3.2	Primitive di <code>Semaphore</code>	30
4	Esecuzione Parallela dei Processi	31
4.1	<i>Thread Pools</i>	31
4.2	Implementazione di <i>Thread Pools</i>	32
4.2.1	Il Metodo <code>newFixedThreadPool</code>	33
4.2.2	La Classe <code>ThreadPoolExecutor</code>	33
4.2.3	La Classe <code>ScheduledThreadPoolExecutor</code>	33
4.2.4	Metodi di <code>ExecutorService</code>	33
5	Conclusioni	35

Capitolo 1

Introduzione

Il seguente Capitolo fornirà un'introduzione teorica al concetto di esecuzione non sequenziale di un processo, ponendo enfasi sui vantaggi da essa portati e sulle nuove necessità richieste da tale implementazione.

1.1 Algoritmo, Programma, Processo

Nella definizione della singola unità di esecuzione si ritiene opportuno effettuare una leggera digressione.

Algoritmo

Un Algoritmo è un procedimento puramente logico studiato appositamente per la risoluzione di uno specifico problema. In letteratura sono rintracciabili moltissimi Algoritmi, per brevità vengono nominati a titolo di esempio Algoritmi di Ordinamento come il *Bubble Sort*[1] - di cui viene fornito lo pseudocodice a seguire - od il *Radix Sort*[2], e Algoritmi di Ricerca del Cammino Minimo in un Grafo come l'*Algoritmo di Dijkstra*[3] o l'*Algoritmo di Bellman-Ford*[4].

Algoritmo 1: Ordinamento per confronti “*Bubble Sort*”

Data: A; // Vettore di numeri interi non ordinati
Result: A; // Vettore di numeri interi ordinati

```
1 partizioneNonOrdinata  $\leftarrow$  A.length;  
2 while partizioneNonOrdinata > 1 do  
3   for i = 2 fino a k do  
4     if A[i] < A[i - 1] then  
5       | Scambia A[i] ed A[i - 1];  
6     end if  
7   end for  
8   partizioneNonOrdinata  $\leftarrow$  (ultimoScambio - 1);  
9 end while  
10 return A
```

Programma

Un Programma è la descrizione in codice di un Algoritmo in uno specifico Linguaggio di Programmazione in maniera tale da permetterne l'esecuzione all'interno di un Calcolatore. Si riporta di seguito un'esempio di Programma che descriva l'Algoritmo di Ordinamento per confronti “*Bubble Sort*” implementato tramite il Linguaggio di Programmazione Java.

```
1 public class BubbleSort {  
2  
3     public static void BubbleSort(int[] A) {  
4  
5         // Definizione della dimensione della partizione del vettore non ordinata  
6         int unorderedPartition = A.length;  
7  
8         // Ordinamento  
9         while(unorderedPartition > 1) {  
10            // Scorro la parte non ordinata  
11            for(int i=2; i<unorderedPartition; i++) {  
12                // Se non e' ordinato, scambia  
13                if(A[i] < A[i-1]) {  
14                    int buf = A[i];  
15                    A[i] = A[i-1];  
16                    A[i-1] = buf;  
17                }  
18            }  
19            // Dopo lo scambio, riduco la dimensione della partizione del vettore non  
20                 $\hookrightarrow$  ordinata  
21            unorderedPartition = unorderedPartition - 1;  
22        }  
23    }  
24 }
```

```
22
23     }
24
25 }
```

Listing 1.1: Esempio di Programma in Linguaggio Java che implementi l'Algoritmo Bubble Sort.

Processo

Un Processo è la sequenza di operazioni eseguite da un Elaboratore che opera sotto il controllo di un Programma: in maniera intuitiva è possibile affermare che in linea generale un Processo è un Programma in esecuzione. Di seguito si riporta un esempio di esecuzione sequenziale di due Processi che eseguono lo stesso Programma, ovvero *Bubble Sort*.

```
$ java BubbleSort 1 2 3 6 5 2
Il vettore non ordinato e': [1, 2, 3, 6, 5, 2]
Il vettore ordinato e': [1, 2, 2, 3, 5, 6]
$ java BubbleSort 1 9 8 3 6 7
Il vettore non ordinato e': [1, 9, 8, 3, 6, 7]
Il vettore ordinato e': [1, 3, 6, 7, 8, 9]
```

Listing 1.2: Due Processi che eseguono il Programma implementante l'Algoritmo Bubble Sort.

1.2 Stati di un Processo

Un Processo può passare attraverso diversi Stati durante il suo ciclo di vita. Nel caso più generale è possibile individuarne essenzialmente tre:

1. **Esecuzione** (o *Running*).

Il Processo detiene il controllo della CPU dell'Elaboratore e sta eseguendo il proprio codice.

2. **Bloccato** (o *Blocked*).

Il Processo non detiene il controllo della CPU dell'Elaboratore e rimane in uno stato dormiente ove attende di essere pronto a richiederne il controllo. Un Processo è in stato *Blocked* tipicamente durante operazioni di I/O mentre è in attesa di dati provenienti da dispositivi meno veloci o dall'Utente.

3. **Pronto** (o *Ready*).

Anche in questo caso il Processo non detiene il controllo della CPU dell'Elaboratore, ma si trova in una coda di Processi che si dicono pronti a richiederne il controllo. Un Processo è in stato *Ready* tipicamente al termine delle suddette

operazioni di I/O mentre attende che il controllo della CPU passi dal Processo attualmente in stato *Running* ad esso.

Le transizioni di Stato che possono avvenire dipendono dall'Algoritmo di *Scheduling* del Sistema Operativo, e sono ben descritte nello schema a seguire.

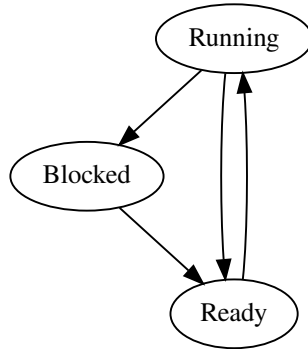


Figura 1.1: Grafo Diretto raffigurante le possibili Transizioni tra i possibili Stati di un Processo. Grafo Generato tramite l'ausilio dell'editor PlantText[5].

Tutte le transizioni di Stato, a meno di quella che porta dallo Stato “*Running*” allo Stato “*Ready*” sono possibili in ogni Algoritmo di Scheduling, mentre quest’ultima è implementabile se e solo se il suddetto Algoritmo supporta meccanismi di Prelazione. Ovviamente la rappresentazione riportata è un caso esemplificativo utile alla comprensione di alcuni comportamenti rintracciabili nella sincronizzazione tra processi nel Linguaggio Java, ma si ricorda che il Grafo degli Stati e delle loro transizioni in un Sistema Operativo moderno risulta nettamente più complicato, si riporta a seguire quello di UNIX.

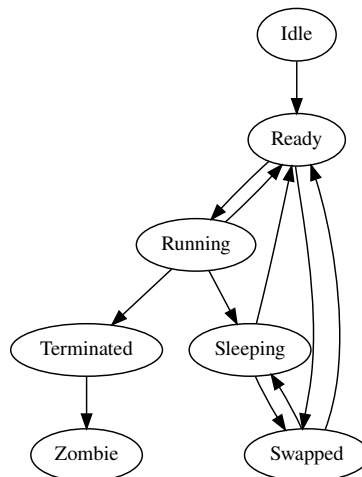


Figura 1.2: Grafo Diretto raffigurante le possibili Transizioni tra i possibili Stati di un Processo in UNIX. Grafo Generato tramite l'ausilio dell'editor PlantText[5].

1.3 Esecuzione Sequenziale ed Esecuzione non Sequenziale

La sequenza di operazioni eseguite da un processo può essere rappresentata attraverso un Grafo Orientato che indichi la precedenza delle une sulle altre. In questa visualizzazione i nodi identificano il singolo evento da verificarsi, mentre gli archi l'ordinamento temporale tra i nodi.

1.3.1 Processi Sequenziali

Facendo riferimento alla rappresentazione descritta in precedenza, un Processo Sequenziale è tale da poter essere schematizzato tramite l'ausilio di un Grafo ad Ordinamento Totale: una particolare tipologia di Grafo ove ogni nodo - e quindi ogni evento - fatta eccezione per il nodo di inizio ed il nodo di fine, possiede uno e un solo predecessore ed uno e un solo successore.

A seguire viene effettuato un esempio di Processo Sequenziale.

Esempio di Rappresentazione di un Processo Sequenziale

Si riporta a titolo di esempio un Processo che richiede all'utente due numeri interi e ne restituisce la somma. Esso è rappresentabile come una sequenza ordinata di operazioni atomiche come esplicitato a seguire.

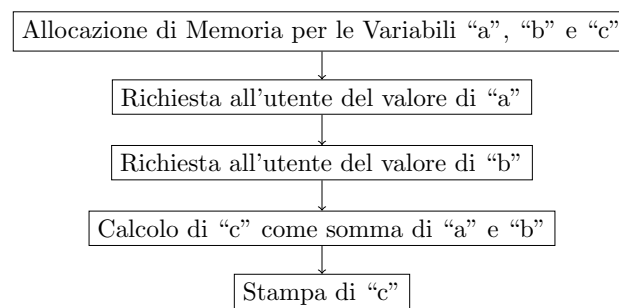


Figura 1.3: Grafo delle Precedenze di un Processo Sequenziale.

1.3.2 Processi non Sequenziali

Al contrario della controparte, un Processo non Sequenziale può essere rappresentato tramite l'ausilio di un Grafo ad Ordinamento Parziale: ogni nodo interno potrà quindi avere più predecessori e più successori. Questo è possibile in quanto alcune operazioni eseguite dal Processo sono indipendenti tra loro o non è interessante il loro ordine di terminazione per l'utente finale.

A seguire viene effettuato un esempio di Processo non Sequenziale.

Esempio di Rappresentazione di un Processo non Sequenziale

Si supponga di dover valutare l'espressione matematica:

$$(5 + 8) + (6 - 3) * (9 + 2)$$

Ovviamente è possibile eseguire sequenzialmente la valutazione verificando prima i risultati delle operazioni tra parentesi, poi la moltiplicazione tra gli ultimi due termini ed infine l'addizione tra i primi due. La natura del problema permette tuttavia di eseguire in ordine arbitrario dei sottoinsiemi di operazioni, i quali risultati risultano indipendenti da un punto di vista di utilizzo dei dati: è possibile in effetti delegare la valutazione di singoli blocchi a differenti agenti di calcolo.

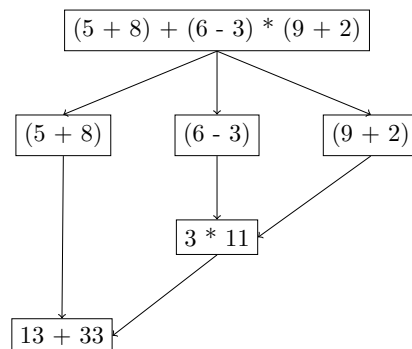


Figura 1.4: Grafo delle Precedenze di un Processo non Sequenziale.

1.4 Processi Interagenti

Dall'esecuzione non sequenziale è possibile ottenere situazioni ove due o più Processi debbano interagire tra loro per il completamento delle rispettive operazioni. È possibile individuare tre possibili tipi di interazione.

1.4.1 Interazione Indiretta (Competizione)

In questo caso i suddetti Processi competono per l'utilizzo di risorse comuni come variabili o dispositivi di Input/Output. Un esempio lampante potrebbe essere la condivisione di una variabile numerica per due Processi P1 e P2.

```
// Variabile Globale  
int x = 10;
```

Esecuzione di P1:

```
...  
x = x + 5;  
...
```

Esecuzione di P2:

```
...  
x = x + 10;  
...
```

Figura 1.5: Esempio di Interazione Competitiva tra due Processi P1 e P2. Entrambi i Processi illustrati vanno ad operare sulla Variabile Globale “x”.

La modifica contemporanea della Variabile Globale “x” da parte di P1 e P2 potrebbe risultare problematica in quanto la lascerebbero in uno stato inconsistente.

Alla base di questa interazione si trova dunque il problema della *Mutua Esclusione*: una risorsa può essere utilizzata da uno e un solo Processo alla volta. Al fine di una corretta esecuzione bisognerà impossibilitare l’esecuzione contemporanea di Operazioni che facciano utilizzo di risorse comuni.

1.4.2 Interazione Diretta (Cooperazione)

La Cooperazione prevede invece lo scambio delle informazioni da un Processo ad un altro. Si prenda in esame la Figura 1.4: il Processo che andrà a valutare l’espressione finale dovrà sicuramente ottenere i risultati del Processo che ha calcolato la prima parentesi e di quello che ha calcolato la moltiplicazione immediatamente successiva. Al fine di preservare la correttezza dell’Operazione si renderà necessario far iniziare le Operazioni del Processo finale successivamente al termine delle Operazioni degli altri due Processi.

1.4.3 Interazione Indesiderata (Interferenza)

Quest’ultima è un effetto indesiderato dato da errori di sincronizzazione tra i processi.

L’Interferenza viene risolta tramite l’utilizzo dei costrutti di sincronizzazione atti alla risoluzione dei problemi nati dalla Competizione e dalla Cooperazione citati in precedenza.

1.5 Sincronizzazione tra Processi

Alla luce di quanto esposto risulta importante esplicitare quali siano gli strumenti e le tecniche di sincronizzazione utilizzate per garantire la correttezza dei risulta-

ti di Processi Concorrenti. Prima di passare ad una loro descrizione è comunque necessario introdurre il concetto di Sezione Critica.

1.5.1 Sezione Critica

La Sezione Critica viene definita come una serie di Operazioni nelle quali un Processo accede e/o modifica risorse comuni a più Processi.

Diverse Sezioni Critiche che accedano a gruppi differenti di Risorse possono essere scisse in altrettante categorie.

L'idea alla base dell'esistenza della Sezione Critica è quella di permettere ad uno ed un solo Processo alla volta di entrare in tale segmento di Codice, in maniera tale da garantire un accesso mutualmente esclusivo alle risorse comuni in essa manipolate.

1.5.2 Strumenti di Sincronizzazione

Gli Strumenti illustrati di seguito non rappresentano l'interezza delle metodologie di Sincronizzazione disponibili in letteratura, ma sono stati scelti in maniera tale da essere il più possibile utili al fine di introdurre i concetti alla base del *Multi-Threading* nel Linguaggio Java ed inoltre chiarificare alcune metodologie utilizzate anche al di fuori di quest'ultimo (si veda lo Scambio di Messaggi).

Lo scopo di tali Meccanismi sarà quello di permettere al più ad un processo di accedere ad ogni categoria di Sezione Critica. Per ottenere questo risultato, la Sezione Critica va incapsulata da un Prologo che la preceda ed un Epilogo che la segua. In maniera intuitiva si può pensare al Prologo e all'Epilogo come l'operazione di acquisizione di un lucchetto per bloccare una risorsa per poi rilasciarlo al prossimo Processo che ne necessita rispettivamente.

A seconda del tipo di Interazione tra i Processi presi in esame è importante definire la corretta metodologia di Sincronizzazione: per Interazioni di tipo Competitivo si utilizzano tipicamente strumenti quali Semafori e Monitor, mentre per Interazioni di tipo Cooperativo si usufruisce della possibilità di scambio di Messaggi tra Processi. Si riportano a seguire i tre Meccanismi citati.

Semaforo

Il Semaforo[6], introdotto dall'informatico olandese Edsger Wybe Dijkstra, è un tipo di dato astratto (ADT) atto alla sincronizzazione che si compone di:

- Valore Intero $S_v \geq 0$;
- Coda di Descrittori Q_s .

E sul quale è possibile effettuare le operazioni di:

- `wait(S)`;
- `signal(S)`.

Il Semaforo va ad incapsulare la Sezione Critica al fine di rendere possibile ad un solo processo alla volta l'accesso ad essa.

La variabile S_v mantiene come informazione quanti processi possono accedere alla loro Sezione Critica (se vale zero il processo si sospende sul Semaforo), mentre Q_s è una Coda di Descrittori dei Processi che non sono riusciti ad accedere alla loro Sezione Critica a causa del Semaforo.

Le Operazioni “wait(S)” e “signal(S)” vanno invece rispettivamente a decrementare ed aumentare il valore della variabile S_v del Semaforo.

L'utilizzo di un Semaforo per la protezione di una Sezione Critica avviene secondo il seguente schema:

Algoritmo 2: Esempio di utilizzo di un Semaforo

```
Data: S;                                     // Semaforo
1 wait(S);
2 Esecuzione della Sezione Critica;
3 signal(S);
```

Viene riportato a titolo di esempio un possibile pseudocodice per l'implementazione di un Semaforo.

Algoritmo 3: Pseudocodice di un Semaforo

```
1 Function Costruttore(int valoreIniziale):  
2   this. $Q_s$  = Nuova Coda di Descrittori di Processo vuota;  
   // Si richiede un valore iniziale da assegnare al Semaforo  
3   this. $S_v$  = valoreIniziale;  
4 Function wait(Semaphore S):  
   // Verifica che il valore di  $S_v$  sia  $> 0$  e decrementalo,  
   altrimenti inserisci il processo nella coda  $Q_s$   
5   if  $S.S_v > 0$  then  
6     |  $S.S_v \leftarrow (S.S_v - 1)$ ;  
7   else  
8     | Imposta lo stato del Processo su “Blocked” ed inserisci il suo descrittore  
     | in  $S.Q_s$ ;  
9   end if  
10 Function signal(Semaphore S):  
   // Verifica che la coda  $Q_s$  non sia vuota e libera l’ultimo  
   processo per poterne riprendere l’esecuzione, altrimenti  
   incrementa il valore di  $S_v$   
11 if  $\exists$  Processo nella Coda  $S.Q_s$  then  
12   | Rimuovi il descrittore del Processo da  $S.Q_s$  ed imposta il suo stato su  
   | “Ready”;  
13 else  
14   |  $S.S_v \leftarrow (S.S_v + 1)$ ;  
15 end if
```

Monitor

Il Monitor[7] è anch’esso un ADT che tiene conto dei Vincoli di Sincronizzazione: implementa infatti da sé meccanismi di Mutua Esclusione per cui al suo interno potrà esserci al più uno e un solo Processo attivo. Una peculiarità nell’utilizzo del Monitor rispetto a quello di un più semplice Semaforo è quella di poter nascondere lo stato interno delle risorse condivise tra i vari processi, lasciando loro visibili unicamente le operazioni effettuabili su di esse.

In maniera più precisa un Monitor “è un costrutto che associa un insieme di procedure ad una struttura dati comune a più Processi” [8]. Esso è composto dai seguenti elementi:

- Costruttore, che genera un’entità di tipo Monitor;
- Variabili Locali, le quali vanno a definire lo stato di un’istanza del tipo Monitor;
- Procedure, per l’interfacciamento con le risorse condivise dai Processi.

Un possibile pseudocodice per un Monitor potrebbe essere simile a quanto segue:

Algoritmo 4: Pseudocodice di un Monitor

```
// Variabili di Condizione
1 Condition variabileCondizione1;
2 Condition variabileCondizione2;
3 ...;
4 Condition variabileCondizioneN;
// Costruttore
5 Function Costruttore():
    | // Codice del Costruttore
// Procedure
6 Procedure Procedura1(Parametro1, Parametro2, ..., ParametroJ):
    | // Codice della Procedura 1
7 Procedure Procedura2(Parametro1, Parametro2, ..., ParametroK):
    | // Codice della Procedura 2
8 ...;
9 Procedure ProceduraN(Parametro1, Parametro2, ..., ParametroL):
    | // Codice della Procedura N
```

Un Monitor dispone di due livelli di Controllo per l'esecuzione esclusiva di operazioni sulle Risorse condivise:

1. **Primo Livello di Controllo:**

Le procedure di un Monitor sono eseguite in maniera Mutualmente Esclusiva, per cui qualora nel Monitor fosse presente un Processo in esecuzione ed un qualsiasi altro processo esterno volesse accedervi, quest'ultimo verrebbe posto in stato "*Blocked*" e si "addormenterebbe" all'esterno di esso.

2. **Secondo Livello di Controllo:**

È possibile per un Processo all'interno di un Monitor di "addormentarsi" su una Variabile di Condizione in quanto potrebbe non rispettare i requisiti imposti. In questo caso quest'ultimo passerà allo stato "*Blocked*" e posto in coda ad essa, così da lasciare spazio ad un altro Processo dall'esterno per poter eseguire.

Le variabili di Condizione di un Monitor dispongono di due primitive "**Wait**" e "**Signal**" che, seppur molto simili a quelle di un Semaforo, godono di leggere differenze: in questo caso infatti la "**Wait**" ha sempre un significato bloccante, mentre la "**Signal**" non produce effetti qualora non ci siano processi in coda.

Scambio di Messaggi

Altro meccanismo di Sincronizzazione tra Processi Interagenti è lo scambio di Messaggi, utile in caso di interazioni di tipo Cooperativo. Per descriverne il funzionamento è importante definire alcune componenti quali le Primitive di Comunicazione “Send” e “Receive” ed il Canale di Comunicazione.

Vista la natura autoesplicativa delle Primitive, si pone ora enfasi sulla definizione del Canale di Comunicazione. Esso deve andare a determinare quali siano le coppie di Processi comunicanti e come identificarle (e quindi scegliere se utilizzare la Struttura Mailbox), se fruire o meno della “Bufferizzazione” e la lunghezza dei messaggi. Si noti che ciascuna di queste decisioni debba - almeno in linea teorica - essere ortogonale alle altre due.

Le scelte implementative sopracitate vengono analizzate a seguire:

- **Tecnica di Identificazione dei Processi Comunicanti**

L’Identificazione dei Processi “Sender” e “Receiver” può avvenire sia direttamente che indirettamente.

Nel primo caso si va ad utilizzare l’identificativo dei processi come parametro delle primitive, come sotto riportato.

```
/*
 * R ed S sono rispettivamente gli Identificatori del Processo "Sender" e del
 *   ↪ Processo "Receiver".
 * La Variabile "message" sta ad indicare il messaggio inviato dal Processo "
 *   ↪ Sender" e che il Processo "Receiver" dovrà ricevere.
 */
send(R, message);
receive(S, message);
```

È anche possibile creare uno schema di tipo “Molti a Uno”, “Uno a Molti” o “Molti a Molti” tramite l’utilizzo di speciali Identificatori all’interno delle primitive “Send” e “Receive”.

```
// Molti a Uno
send(R, message); // Invia ad uno specifico Ricevitore
receive(ID, message); // Riceve da qualsiasi Mittente
// Uno a Molti
send(setOfReceivers, message); // Invia a piu' Ricevitori
receive(S, message); // Riceve dallo specifico Mittente "S"
// Molti a Molti
send(setOfReceivers, message); // Invia a piu' Ricevitori
receive(ID, message); // Riceve da qualsiasi Mittente
```

Per implementare un tipo di Identificazione Indiretta si va ad interporre un interlocutore terzo - la “Mailbox” - che si occupa di permettere ai Mittenti di inviare lei un messaggio, il quale verrà letto dai Destinatari a lei collegati.

- **Utilizzo della tecnica di “*Bufferizzazione*”**

L'utilizzo di tale funzionalità permette una comunicazione asincrona che consente un maggiore parallelismo all'applicativo. Il “*Buffer*” può essere di dimensione limitata o infinita. È possibile l'utilizzo di Messaggi di “*Acknowledgement*” - o ACK - da parte del Processo “*Receiver*” per confermare la ricezione del Messaggio. La corretta sequenza di ricezione potrebbe non essere garantita in caso di *Routing* Dinamico dei Messaggi, per cui si rende necessario l'utilizzo di una numerazione per l'ordinamento.

- **Scelta della lunghezza dei Messaggi**

È infine possibile scegliere se implementare dei Messaggi a lunghezza fissa, rendendo l'implementazione semplice ma spreco di memoria; o utilizzare Messaggi di lunghezza variabile, che risolvono il problema dei primi al costo di una più complessa realizzazione. Nei Sistemi Operativi moderni viene in linea generale adottato questo secondo approccio: ad esempio in UNIX vengono utilizzati messaggi di lunghezza variabile con dimensione minima di 1 Byte.

1.5.3 Modelli Logici per la Concorrenza

Nel tempo sono stati definiti due Modelli Logici di riferimento per la gestione concorrente dei Processi: il Modello ad Ambiente Globale ed il Modello ad Ambiente Locale.

Modello ad Ambiente Globale

In tale contesto un'applicazione viene descritta come un insieme di Processi e Risorse, ove queste ultime sono una qualsiasi entità fisica o logica di cui uno o più Processi dell'applicazione stessa necessitano per portare a termine la loro esecuzione.

Data la natura condivisa di questo Ambiente sono possibili interazioni sia competitive che cooperative per l'utilizzo e lo scambio di risorse tra i Processi.

Modello ad Ambiente Locale

Definito successivamente, in questo Modello un'applicazione viene descritta come un insieme di Processi, ognuno dei quali va ad operare nel proprio ambiente utilizzando le proprie risorse, inaccessibili agli altri Processi della stessa applicazione.

La diversa concezione porta alla semplificazione delle interazioni tra i processi, rendendo possibili unicamente quelle di tipo cooperativo.

1.6 Processi Pesanti e Processi Leggeri

Un Sistema Operativo può supportare due diverse tipologie di Processi, le quali vengono di seguito descritte.

1.6.1 Processi Pesanti

I Processi Pesanti dispongono di un loro Program Counter, di un loro stato dei vari Registri della CPU, di un proprio *Stack*, del proprio Codice e dei Propri Dati, per questo vengono identificati tramite la ennupla:

$$\textit{Processo_Pesante} = \{\textit{Program Counter}, \textit{Registri}, \textit{Stack}, \textit{Codice}, \textit{Dati}\}$$

Com'è facilmente intuibile Processi Pesanti distinti eseguono Codice distinto ed accedono a Dati distinti, non condividendo memoria ed ispirandosi quindi al Modello ad Ambiente Locale. La comunicazione tra più Processi pesanti avviene tramite l'utilizzo di tecniche quali lo scambio di Messaggi data la natura cooperativa delle loro interazioni e la condivisione di risorse è complessa, richiedendo una porzione di Memoria condivisa.

1.6.2 Processi Leggeri - “*Thread*”

I Processi Leggeri - o *Thread* - sono invece identificati dalla ennupla:

$$\textit{Thread} = \{\textit{Program Counter}, \textit{Registri}, \textit{Stack}\}$$

Più *Thread* correlati da un punto di vista logico condividono un'area di dati e codice tra loro formando un *Task* e ciascuno di essi rappresenta differenti contesti di esecuzione che risultano indipendenti all'interno dello stesso. Da ciò segue che i *Task* sono invece identificati da una ennupla del tipo:

$$\textit{Task} = \{\textit{Thread}_1, \textit{Thread}_2, \dots, \textit{Thread}_n, \textit{Codice}, \textit{Dati}\}$$

Alla luce di quanto illustrato è possibile immaginare un Processo Pesante come un *Task* composto da un solo *Thread*.

Si noti come sia comunque necessario, ai fini del *Context Switching*, mantenere un Descrittore di Processo (*Process Control Block* - *PCB*) per ogni Processo Leggero, rendendo più complesso il *PCB* del *Task* che va ad incorporare anche quelli dei suoi relativi *Thread*. L'implementazione di Processi Leggeri permette tuttavia operazioni di Cambio di Contesto all'interno dello stesso *Task* nettamente più veloci della controparte, in quanto i *PCB* dei singoli *Thread* risulta molto meno verboso di quello di

un singolo Processo Pesante non contenendo alcuna informazione riguardo Codice e Dati.

In termini implementativi, i *Thread* di uno stesso *Task* condividono lo spazio di indirizzamento, le variabili globali e la tabella dei File aperti proprie del *Task*, ispirandosi quindi al Modello ad Ambiente Globale. Questa somiglianza implica l'insorgenza di problematiche quali le cosiddette “*Race Conditions*” date da interazioni di tipo sia Competitivo che Cooperativo: analogamente a quanto anticipato in precedenza, andranno di conseguenza implementate misure atte alla sincronizzazione tra i vari Processi Leggeri del *Task*.

Modalità di Realizzazione

I *Thread* possono essere implementati su due distinti livelli:

1. Livello Kernel

Il Sistema Operativo gestisce autonomamente l'avvicinarsi dei vari Processi Leggeri all'interno dell'Elaboratore, mettendo inoltre a disposizione i vari strumenti di sincronizzazione precedentemente analizzati.

2. Livello Utente

In questo caso il Sistema Operativo vede unicamente l'intero *Task* come un comune Processo Pesante, quindi il Cambio di Contesto tra *Thread* di uno stesso *Task* avviene senza interrompere il Sistema Operativo. Questo vantaggio viene controbilanciato dalla necessità di bloccare l'intero *Task* alla chiamata di una *Routine* bloccante all'interno di un singolo *Thread*.

Ed il Sistema Operativo può supportare diversi Modelli Implementativi:

- **One-to-One**

Ogni *Thread* Utente viene mappato ad un solo *Thread Kernel*.

- **Many-to-One**

Più *Thread* Utente vengono mappati ad un solo *Thread Kernel*.

- **Many-to-Many**

Più *Thread* Utente vengono mappati a più *Thread Kernel*.

Capitolo 2

Thread in Java

Il seguente Capitolo tratterà l'implementazione di un Processo Leggero nel Linguaggio di Programmazione Java.

2.1 Creazione di un Processo Leggero

Ogni Processo Leggero in Java viene rappresentato da un Oggetto di Tipo **Thread** o di un suo Sotto-Tipo: questa semantica permette di creare Classi che possano essere eseguite su *Thread* separati dal principale, permettendo quindi un migliore parallelismo nell'esecuzione delle loro operazioni.

Java mette a disposizione del Programmatore due diverse possibilità per la stesura di un Programma che utilizzi più Processi Leggeri: l'estensione della Classe **Thread** e l'implementazione dell'Interfaccia **Runnable**.

Prima di descrivere entrambe le opportunità è bene fornire informazioni sull'ereditarietà della Classe **Thread**: essa infatti estende la Classe **Object** ed implementa l'interfaccia **Runnable** così come raffigurato nel Diagramma UML di seguito.

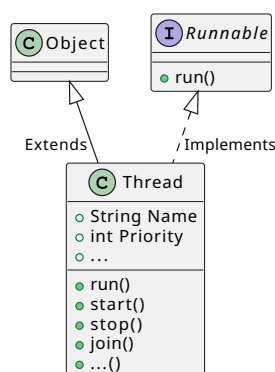


Figura 2.1: Ereditarietà della Classe **Thread** in Java. UML Generato tramite l'ausilio dell'editor PlantText[5].

Come affermato in precedenza, l'obiettivo ultimo di un Programma che utilizzi più Processi Leggeri è quello di avere un oggetto di tipo **Thread** (o di un suo Sotto-Tipo) per ognuno di essi, così da sfruttarne le Primitive. Si illustrano ora le due diverse possibilità di stesura di un Programma Multiprocesso.

2.1.1 Estensione della Classe Thread

Un primo approccio alla Programmazione *Multi-Threaded* in Java può essere quello di Estendere la Classe **Thread** con la propria.

L'unico requisito è quello di completare l'implementazione del Metodo **run()** della Super-Classe esprimendo le operazioni da eseguire una volta iniziata l'esecuzione del nuovo *Thread*.

Si riporta a seguire un esempio didattico per l'utilizzo dove il Thread principale andrà a creare un numero prestabilito di Processi Leggeri, i quali dovranno stampare sullo Standard Output un messaggio con l'identificativo loro assegnato in fase di inizializzazione per poi terminare l'esecuzione.

```
1 public class ClassExtendThread extends Thread {
2
3     // Definisco l'Identificativo che verra' assegnato dal Main
4     private int threadNumber;
5
6     public ClassExtendThread(int tNum) {
7         // Inizializzo l'Identificativo
8         this.threadNumber = tNum;
9     }
10
11     // Implemento il metodo run() sovrascrivendo l'implementazione di Thread
12     @Override
13     public void run() {
14         // Stampo l'Identificativo
15         System.out.println("Sono il Thread numero " + this.threadNumber + ".");
16     }
17 }
```

Listing 2.1: Esempio di Classe che estende la Classe Thread per essere eseguita in un Processo Leggero separato.

Nel Main verrà creato un oggetto di tipo **ClassExtendThread** per ogni Processo Leggero, per poi utilizzarne la primitiva **start()** ereditata da **Thread**.

```
1 public class MainExtendThread {
2
3     // Definisco a priori il numero di Thread da creare
4     private static final int NUM_OF_THREADS = 5;
5
6     public static void main(String[] args) {
```

```

7
8      System.out.println("Sono il Main e sto per creare i Thread.");
9
10     // Creo NUM_OF_THREADS Thread
11     for(int i=0; i<NUM_OF_THREADS; i++) {
12         ClassExtendThread cet = new ClassExtendThread(i);
13         // Utilizzo la primitiva start di Thread
14         cet.start();
15     }
16
17     // Avviso l'utente della terminata esecuzione del Main
18     System.out.println("Sono il Main ed ho terminato l'esecuzione.");
19 }
20 }

```

Listing 2.2: Classe Main per l'utilizzo della Classe precedentemente illustrata.

2.1.2 Implementazione dell'Interfaccia Runnable

Alternativa alla modalità precedente è quella di implementare l'Interfaccia `Runnable`.

Tale cambiamento non modificherà in maniera radicale il codice, ma necessiterà di un differente ragionamento: ora infatti andrà creato un oggetto di tipo `Thread` a partire da un oggetto della Classe implementante `Runnable`, per poi operare direttamente sull'oggetto di tipo `Thread`.

Si riporta a seguire una reimplementazione dell'esempio precedente seguendo questa nuova logica.

```

1 public class ClassImplementRunnable implements Runnable {
2     ...
3 }

```

Listing 2.3: Modifica effettuata al precedente esempio per adattare il codice al nuovo approccio.

Com'è possibile notare dal Codice sopra le modifiche all'interno della Classe saranno minime: i requisiti sono infatti esattamente gli stessi. Il vero cambiamento si avrà nella classe principale che dovrà gestire come segue la creazione dei nuovi Processi Leggeri.

```

1 ...
2 for(int i=0; i<NUM_OF_THREADS; i++) {
3     // Creo un nuovo oggetto di tipo ClassImplementRunnable
4     ClassImplementRunnable cir = new ClassImplementRunnable(i);
5     // Creo un Thread a partire dall'oggetto della mia Classe
6     Thread thread = new Thread(cir);
7     // Eseguo il Thread

```

```
8 |     thread.start();
9 | }
10| ...
```

Listing 2.4: Classe Main per l'utilizzo della Classe precedentemente illustrata.

2.1.3 Considerazioni Pratiche ed Esecuzione

Le due implementazioni, seppur molto simili visivamente, risultano differenti da un punto di vista logico per le motivazioni indicate poc'anzi. Per ragioni pratiche legate al fatto che in Java non risulta possibile estendere più di una Classe alla volta ma è possibile implementare più Interfacce contemporaneamente, si preferisce generalmente procedere con il secondo approccio.

L'esecuzione dei due Programmi così come sono stati stesi risulterà nel medesimo risultato: una stampa in ordine randomico da parte di ciascuno dei Processi Leggeri invocati.

```
$ java MainImplementRunnable
Sono il Main e sto per creare i Thread.
Sono il Main ed ho terminato l'esecuzione.
Sono il Thread numero 2.
Sono il Thread numero 3.
Sono il Thread numero 0.
Sono il Thread numero 4.
Sono il Thread numero 1.
```

Listing 2.5: Esempio di esecuzione dei Programmi sopra riportati.

Si noti come l'esecuzione del *Thread* principale termini prima di quella degli altri Processi Leggeri da lui generati ma che questa circostanza non limiti in alcun modo questi ultimi, che continueranno la loro esecuzione fino alla rispettiva terminazione.

2.2 Primitive per la Gestione dell'Esecuzione

La classe **Thread** mette a disposizione Metodi per la gestione dell'Esecuzione dei singoli Processi Leggeri[9]. Ciò risulta utile nel caso in cui si voglia influenzare il loro ordine di esecuzione o ancora la loro possibilità di eseguire.

Il metodo `start()`

Il metodo `start()` permette di inizializzare l'esecuzione del Processo Leggero invocando il metodo `run()` della Classe.

Il metodo `stop()`

Il metodo `stop()` permette di interrompere invece l'esecuzione del Processo Leggero sollevando un'Eccezione di tipo `ThreadDeath`.

Il metodo `suspend()`

Il metodo `suspend()` permette di sospendere momentaneamente l'esecuzione di un *Thread* fin tanto che su di lui non venga chiamato il metodo `resume()`, portandolo dallo stato *Running* allo stato *Blocked*.

Il metodo `resume()`

Il metodo `resume()` permette di riprendere l'esecuzione di un *Thread* bloccato dal metodo `suspend()`, portandolo dallo stato *Blocked* allo stato *Ready*.

Il metodo `sleep(int millis)`

Il metodo `sleep()` permette di portare il Processo Leggero nello stato *Blocked* per un determinato numero di millisecondi senza utilizzare cicli di Clock della CPU. Tale metodo solleva un'Eccezione di tipo `InterruptedException` nel caso in cui a tale Processo Leggero venisse chiesto di interrompersi mentre è già in stato *Blocked*, perciò il metodo va incapsulato in un costrutto di tipo *Try-Catch*.

Il metodo `destroy()`

Il metodo `destroy()` permette di eliminare il *Thread* senza effettuare alcuna pulizia, mantenendo quindi i vari *Lock* da esso utilizzati e facilitando l'insorgere di una situazione di Blocco Critico in quanto nessun altro Processo potrà più accedere a tali risorse.

Il metodo `yield()`

Il metodo `yield()` permette al *Thread* di lasciare volontariamente la CPU ad un altro processo, implicando un successivo *Re-Scheduling* dei Processi da eseguire.

Il metodo `join()`

Il metodo `join()` consente al Processo Padre di attendere la terminazione del Processo Figlio prima di continuare con la propria esecuzione, garantendo un semplice meccanismo di sincronizzazione tra i due.

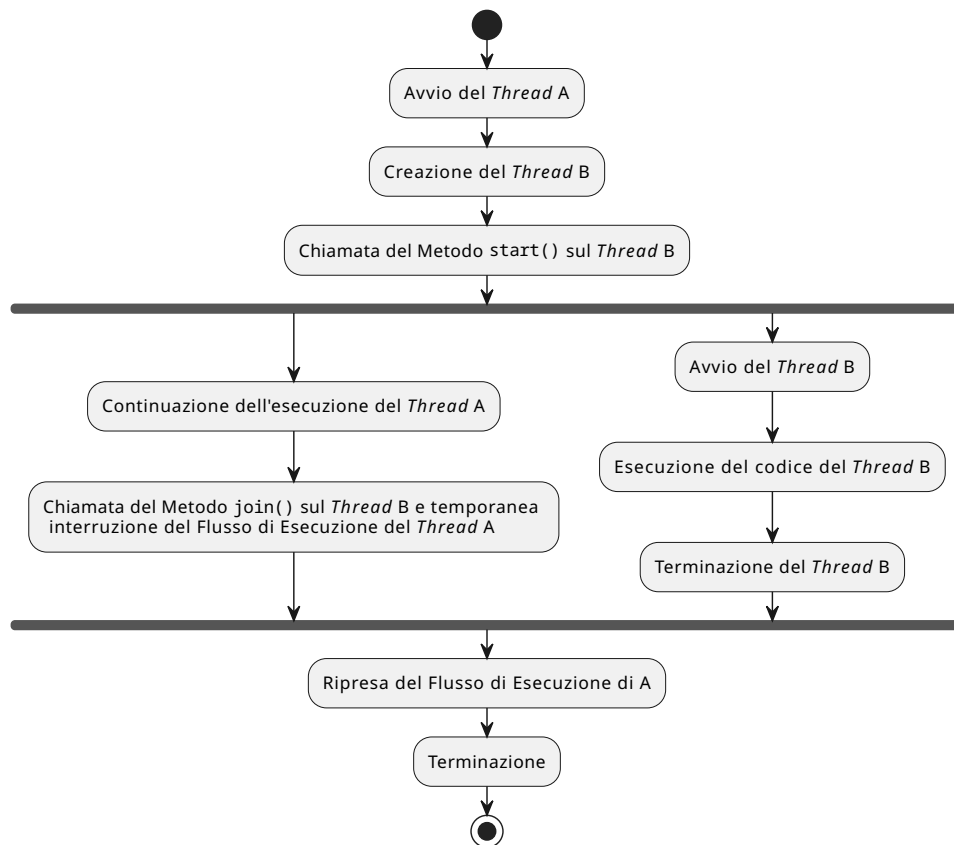


Figura 2.2: Flusso di Esecuzione di un Processo Padre ed un Processo Figlio nel caso di utilizzo del Metodo `join()`. Grafo Generato tramite l'ausilio dell'editor PlantText[5].

2.2.1 Considerazioni

Come discusso in precedenza, l'arbitraria interruzione dell'esecuzione di un Processo potrebbe far incorrere in situazioni indesiderate: le risorse da lui utilizzate potrebbero rimanere infatti in uno stato inconsistente o indefinitivamente bloccate se tale Processo aveva acquisito un *Lock* per l'accesso esclusivo a tale risorsa facilitando l'insorgere di situazioni di Blocco Critico, infine la possibilità di bloccare per un tempo arbitrario l'esecuzione di un *Thread* potrebbe portare alla *Starvation* di questo.

Per questi motivi i Metodi `stop()`, `suspend()`, `resume()` e `destroy()` sono stati deprecati a partire da Java 1.2 nel 1998 e se ne scoraggia fortemente l'utilizzo.

Alle primitive sopracitate si preferisce usufruire della capacità di segnalazione Inter-Processo comunicando la possibilità di sospendere o terminare la loro esecuzione, in maniera tale da permettere loro di eseguire le proprie Sezioni Critiche e lasciare il Sistema in uno stato consistente. Un esempio di Metodo che permette al Processo Leggero di lasciare volontariamente la CPU è `yield()`. La natura delle moderne versioni della *Java Virtual Machine* implica poi l'utilizzo dello *Scheduler* del Sistema

Operativo, il quale dovrà scegliere il prossimo Processo cui lasciare il controllo della CPU. Notando che tutti i moderni Algoritmi di *Scheduling* supportano l'utilizzo di un valore di Priorità per la scelta del Processo dalla Coda dei Processi Promti, anche Java mette a disposizione un valore di Priorità per i propri Processi Leggeri: esso varia tra il valore minimo "MIN_PRIORITY" (0) ed il valore massimo "MAX_PRIORITY" (10) e viene impostato in maniera automatica - al netto di ulteriori istruzioni - su "NORM_PRIORITY" (5), risultando visualizzabile e personalizzabile rispettivamente tramite le Primitive `getPriority()` e `setPriority()`. Tra le altre primitive utilizzate per una migliore sincronizzazione tra i processi risulta indubbiamente presente il Metodo `join()` che permette al Processo Padre di attendere la terminazione del Processo Figlio prima di continuare con la propria Esecuzione.

Capitolo 3

Sincronizzazione tra Processi

Nel Capitolo che segue verranno discusse alcune delle tecniche di Sincronizzazione tra *Thread* messe a disposizione dal Linguaggio di Programmazione Java.

3.1 Il Costrutto *Synchronized*

Il Costrutto *Synchronized* permette di implementare, come il nome suggerisce, una sincronizzazione nell'esecuzione di Metodi o comunque di blocchi di codice da parte di due o più Processi Leggeri. Essi si basano sull'utilizzo di Oggetti che svolgeranno la funzione di Monitor, garantendo ad un solo Processo alla volta di eseguire le parti di codice protette di un determinato oggetto.

È possibile utilizzare la parola chiave **synchronized** sia nella dichiarazione di un Metodo che nella protezione di uno specifico blocco di codice, sono riportati a seguire esempi implementativi.

```
public synchronized void methodName()  
    ↪ {  
    // Metodo Protetto  
}
```

(a) Esempio di dichiarazione di un Metodo *Synchronized*.

```
synchronized(Object) {  
    // Blocco di Codice Protetto  
}
```

(b) Esempio di dichiarazione di un Blocco di Codice *Synchronized*.

Figura 3.1: Esempio di utilizzo del Costrutto *Synchronized*.

3.1.1 Metodi *Synchronized*

Il Monitor dei Metodi di tipo *Synchronized* è implicito, e può essere una tra le seguenti due alternative:

- L'Oggetto **this** (l'Istanza della Classe) nel caso di Metodi d'Istanza;
- La Classe stessa nel caso di Metodi Statici.

Questo vuol dire che più Metodi all'interno di una classe potrebbero essere gestiti tramite lo stesso Monitor, garantendovi un accesso mutualmente esclusivo.

3.1.2 Blocchi *Synchronized*

Nei Blocchi *Synchronized*, per natura di dichiarazione, la Variabile utilizzata come Monitor non è implicita. In questo caso è possibile quindi definire diverse “Classi di Mutua Esclusione”. Grazie a questa flessibilità è anche possibile condividere l'oggetto Monitor tra più Istanze della stessa Classe o anche tra più Classi.

3.1.3 Le Primitive `wait()`, `notify()` e `notifyAll()`

È possibile permettere ai vari *Thread* di “addormentarsi” e “risvegliarsi” all'interno di un Blocco *Synchronized* per garantire la corretta esecuzione del Programma. Java mette a disposizione tre primitive.

`wait()`

Permette al *Thread* chiamante di interrompere la propria esecuzione e rimanere in stato *Blocked* all'interno del Monitor, consentendo ad un altro Processo Leggero di accedere al Codice Protetto.

`notify()`

Permette al *Thread* chiamante di risvegliare uno dei *Thread* in attesa di eseguire il Codice Protetto dal Monitor. Viene utilizzato generalmente all'uscita dalla sezione di Codice Protetto.

`notifyAll()`

Permette al *Thread* chiamante di risvegliare tutti i *Thread* in attesa di eseguire il Codice Protetto dal Monitor. Anch'esso viene utilizzato generalmente all'uscita dalla sezione di Codice Protetto.

3.1.4 Rientranza del Costrutto *Synchronized*

Il Costrutto *Synchronized* si dice Rientrante in quanto qualora un Thread eseguente un blocco di codice sincronizzato volesse eseguirne un altro controllato tramite lo stesso Monitor, gli verrà concessa tale opportunità. Si riporta a seguire un esempio di Programma e della sua relativa esecuzione al fine di illustrare tale proprietà.

```
1 public class SynchronizedReentrance {  
2  
3     public static void main(String[] args) {
```

```

4
5      // Creo un'istanza della Classe di Supporto con i metodi Synchronized
6      SynchronizedReentranceSupportClass sc = new
          ↪ SynchronizedReentranceSupportClass();
7
8      // Creo due Thread utilizzando una Espressione Lambda
9      Thread t1 = new Thread(() -> {
10          for(int i=0; i<5; i++) {
11              sc.increaseAndPrintCounter(1);
12          }
13      });
14      Thread t2 = new Thread(() -> {
15          for(int i=0; i<5; i++) {
16              sc.increaseAndPrintCounter(2);
17          }
18      });
19
20      // Avvio i due Thread
21      t1.start();
22      t2.start();
23  }
24 }
25
26 class SynchronizedReentranceSupportClass {
27
28     // Inizializzo una variabile contatore
29     private int counter = 0;
30
31     // Il metodo verra' raggiunto tramite l'altro esposto
32     private synchronized void increaseCounter() {
33         counter++;
34     }
35
36     // Metodo esposto ai Thread
37     protected synchronized void increaseAndPrintCounter(int threadNum) {
38         this.increaseCounter();
39         System.out.println("Il contatore vale " + this.counter + " ed e' stato
          ↪ aggiornato dal Thread " + threadNum);
40     }
41 }

```

Listing 3.1: Programma che sfrutti la proprietà di Rientranza del Costrutto `synchronized`.

```

$ java SynchronizedReentrance
Il contatore vale 1 ed e' stato aggiornato dal Thread 1
Il contatore vale 2 ed e' stato aggiornato dal Thread 1
Il contatore vale 3 ed e' stato aggiornato dal Thread 1

```

```
Il contatore vale 4 ed e' stato aggiornato dal Thread 1
Il contatore vale 5 ed e' stato aggiornato dal Thread 1
Il contatore vale 6 ed e' stato aggiornato dal Thread 2
Il contatore vale 7 ed e' stato aggiornato dal Thread 2
Il contatore vale 8 ed e' stato aggiornato dal Thread 2
Il contatore vale 9 ed e' stato aggiornato dal Thread 2
Il contatore vale 10 ed e' stato aggiornato dal Thread 2
```

Listing 3.2: Esecuzione del Programma.

3.1.5 Considerazioni

A fronte di una semplice implementazione, l'utilizzo del Costrutto `synchronized` porta prestazioni non desiderabili in quanto un solo *Thread* alla volta può accedere ad un Blocco di Codice controllato dallo stesso Oggetto. Altro svantaggio è quello di non avere una garanzia sulla sequenza di esecuzione dei vari Thread, quindi è possibile incorrere in fenomeni di *Starvation*, il Costrutto infatti non è Equo.

3.2 L'Interfaccia Lock

Le diverse implementazioni di `Lock` forniscono un'alternativa per la gestione delle Sezioni Critiche.

Il concetto alla base è analogo a quello dell'alternativa precedente: quando un *Thread* ha acquisito un *Lock*, nessun altro può eseguire operazioni protette da questo fin tanto che esso non viene rilasciato. Un tipico schema di utilizzo viene riportato a seguire.

```
Lock lck = new ...;
try {
    // Acquisizione del Lock
    lck.lock();
    // Sezione Critica
    ...
    // Fine Sezione Critica
} catch(Exception e) {
    // Gestione delle (eventuali) Eccezioni sollevate
    e.printStackTrace();
} finally {
    // Rilascio del Lock
    lck.unlock();
}
```

Listing 3.3: Esempio di utilizzo di un *Lock*.

Vista la criticità del *Lock* è buona norma operare su di esso all'interno di un Blocco *Try-Finally* o *Try-Catch* in maniera tale da rilasciarlo anche nel caso di Eccezioni e permettere la normale esecuzione di altri *Thread*.

3.2.1 Categorie di *Lock*

Essendo *Lock* solamente un'interfaccia, ne sono disponibili diverse tipologie che implementino la stessa funzionalità estendendola poi in maniera differente. A seguire le due categorie più utilizzate.

ReentrantLock

Il *Lock* Rientrante è posseduto dall'ultimo Processo Leggero che è riuscito a bloccarlo e non lo ha ancora rilasciato. È possibile per un *Thread* acquisire lo stesso *Lock* più volte, rilasciandolo poi un numero di volte pari a quello per cui è stato acquisito al fine di renderlo disponibile ad altri Processi Leggeri.

ReentrantReadWriteLock

Alternativa particolarmente utile nel caso di consumazione di risorse in modalità distinte di Lettura o Scrittura che implementa una semantica simile al *ReentrantLock* illustrato poc'anzi. Permette di definire un "Sotto-*Lock*" di Lettura ed un altro di Scrittura.

3.2.2 Equità del *Lock*

A differenza della controparte è possibile imporre al *Lock* di essere equo e permettere di essere acquisito dai *Thread* nell'ordine di arrivo, prioritizzando quindi quelli che stanno attendendo la sua acquisizione da più tempo ed evirando l'insorgere di problematiche relative alla *Starvation* di un *Thread*.

Per imporre questa specifica è unicamente necessario utilizzare il corretto costruttore del *Lock* desiderato, passando come parametro un Booleano impostato su **true** come segue.

```
// Lock non equi
Lock unfairLock1 = new ReentrantLock();
Lock unfairLock2 = new ReentrantLock(false);
// Lock equo
Lock fairLock = new ReentrantLock(true);
```

Listing 3.4: Esempi di inizializzazione di *Lock* sia equi che non.

3.2.3 Primitive di Lock

L'interfaccia mette a disposizione due Metodi al di fuori dei già discussi `lock()` ed `unlock()`:

`lockInterruptibly()`

Questo Metodo non permetterà di acquisire il *Lock* qualora il *Thread* sia stato interrotto sollevando un'Eccezione di tipo `InterruptedException`.

`tryLock()`

Questo Metodo acquisisce il *Lock* solamente se disponibile al momento dell'invocazione, altrimenti ritorna valore un Booleano `false` e permette al *Thread* di continuare normalmente con la sua esecuzione, non è quindi bloccante a differenza del normale `lock()`.

Ne esiste una variante in cui viene passato come parametro un valore numerico rappresentante il tempo di tolleranza ed un altro parametro che indica l'unità di misura del tempo indicato precedentemente: il *Thread* proverà ad acquisire il *Lock* per il tempo specificato verificando che il Processao Leggero che lo possiede non sia stato interrotto prima di ritornare e continuare con il suo flusso di Esecuzione.

3.2.4 Variabili Condizione

Le Variabili di Condizione vanno a sostituire l'Oggetto Monitor e le sue primitive (`wait()`, `notify()` e `notifyAll()`) dando l'impressione di avere a disposizione più insiemi di condizioni per la sincronizzazione e vengono utilizzate in congiunzione con le varie implementazioni di *Lock*. Esse permettono quindi di sospendere e riprendere l'esecuzione di un Processo Leggero sotto determinate ipotesi all'interno di una Sezione Critica descritta dal *Lock* collegato.

Inizializzazione di una Variabile di Condizione

Il legame intrinseco con il *Lock* associato di una Variabile Condizione implica che la sua dichiarazione avverrà per mezzo di quest'ultimo. Se ne riporta a seguire un esempio.

```
Lock lck = new ReentrantLock();  
Condition cond = lck.newCondition();
```

Listing 3.5: Dichiarazione di una Variabile Condizione `cond` legata al Lucchetto `lck`.

Primitive delle Variabili Condizione

Analogamente a quanto trovato nella descrizione del Costrutto `synchronized`, le Variabili Condizione dispongono di primitive per la sospensione e la ripresa dell'esecuzione di un *Thread*. Esse sono rispettivamente `await()` e `signal()` (quest'ultimo disponibile anche nella sua variante `signalAll()`). Ancora una volta per la Primitiva `await()` è disponibile una variante che accetta come parametro un tempo massimo di attesa prima della ripresa del Flusso di Esecuzione del *Thread*, nonché una diversa Primitiva - `awaitUninterruptibly()` - che sospende l'esecuzione del Processo fino alla segnalazione senza però interromperlo.

3.3 La Classe Semaphore

Java dispone di una Classe rappresentante un Semaforo Generalizzato: essa permette di limitare il numero di Processi Leggeri che possono accedere contemporaneamente ad una determinata risorsa.

3.3.1 Equità di un Semaforo

La dichiarazione di un Semaforo può passare attraverso due diversi costruttori: uno che indichi unicamente il numero di *Thread* che possono richiedere di entrare contemporaneamente nella Sezione Critica da esso protetta, ed un altro che oltre a questo valore accetta anche una Variabile Booleana che ne indichi l'Equità, permettendo quindi anche in questo caso di garantire al Processo da più tempo in attesa sul Semaforo di ottenere il permesso di eseguire prima degli altri. Si riporta a seguire un esempio di dichiarazione di varie tipologie di Semafori.

```
// Variabile di Supporto per il numero di massimi permessi disponibili
int maxPermits = ...;
// Semafori Mutualmente Esclusivi (non Equi)
Semaphore nonFairMutex1 = new Semaphore(1);
Semaphore nonFairMutex2 = new Semaphore(1, false);
// Semafori Generalizzati (non Equi)
Semaphore nonFairGeneralized1 = new Semaphore(maxPermits);
Semaphore nonFairGeneralized2 = new Semaphore(maxPermits, false);
// Semaforo Mutualmente Esclusivo (Equo)
Semaphore fairMutex = new Semaphore(1, true);
// Semaforo Generalizzato (Equo)
Semaphore fairGeneralized = new Semaphore(maxPermits, true);
```

Listing 3.6: Esempi di dichiarazione dei diversi tipi di Semaforo in Java.

3.3.2 Primitive di Semaphore

Nuovamente la Classe dispone di diverse primitive per la gestione dell'esecuzione della Sezione Critica da parte dei vari *Thread*. Si riportano a seguire le più utilizzate.

`acquire()`

Permette di acquisire uno (o `numberOfPermits` nella sua variante `acquire(int numberOfPermits)`) permessi per l'accesso alla Sezione Critica.

`acquireUninterruptibly()`

Anch'esso permette al *Thread* di acquisire uno o `numberOfPermits` nella variante `acquireUninterruptibly(int numberOfPermits)` permessi per l'accesso alla Sezione Critica, ma senza essere effettivamente interrotto dallo *Scheduler*.

`tryAcquire()`

Utile nel tentare l'acquisizione di uno o più permessi nella sua variante `tryAcquire(int numberOfPermits)` in maniera non bloccante: il *Thread* tenterà di acquisire il permesso, e se questo non è disponibile esso proseguirà con la propria esecuzione. Ne esiste una variante temporizzata (`tryAcquire(int numberOfPermits, long time, TimeUnit unit)`) che permette di provare ad acquisire il permesso per un lasso di tempo pari a quello specificato.

`release()`

Il metodo consente di rilasciare uno o `numberOfPermits` nella sua variante `release(int numberOfPermits)` permessi, consentendo al prossimo Processo Leggero di acquisirli.

`reducePermits(int numberOfPermitsToRemove)`

È anche possibile ridurre il numero di permessi allocabili dal Semaforo utilizzando questa primitiva, in maniera tale da ridurre gli accessi concorrenti possibili alla Sezione Critica da esso protetta.

`availablePermits()`

Ritorna il numero di Permessi ancora disponibili e non utilizzati da alcun *Thread* all'interno della Sezione Critica protetta dal Semaforo.

Capitolo 4

Esecuzione Parallela dei Processi

Il Linguaggio di Programmazione Java permette di semplificare ed ottimizzare l'esecuzione parallela di differenti *Task* mettendo a disposizione del Programmatore il Costrutto delle *Thread Pool*, illustrato nel presente Capitolo.

4.1 *Thread Pools*

Si tratta di gruppi di *Thread* cosiddetti “*Worker*” che attendono l'allocazione di *Task* da eseguire.

Questo approccio è spesso utilizzato in quanto l'allocazione di un *Thread* richiede un *Overhead* di risorse che potrebbe divenire non banale con il crescere del numero di questi. Altro vantaggio portato da una filosofia che privilegi il riutilizzo di *Thread* è quella di poter limitare il tempo utilizzato per la loro creazione e distruzione.

Non essendo però più necessariamente in grado di assegnare un singolo *Thread* ad ogni *Task* dell'utente, il grado di parallelismo raggiungibile implementando una soluzione come questa va ovviamente a ridursi.

Ogni *Thread Pool* dispone di una lista di *Task* da eseguire da cui ciascun *Thread* andrà a scegliere. Una volta che un *Thread* termina l'esecuzione di un *Task*, esso andrà a sceglierne un altro dalla lista fin tanto che ve ne saranno presenti. A seguire è illustrato un esempio.

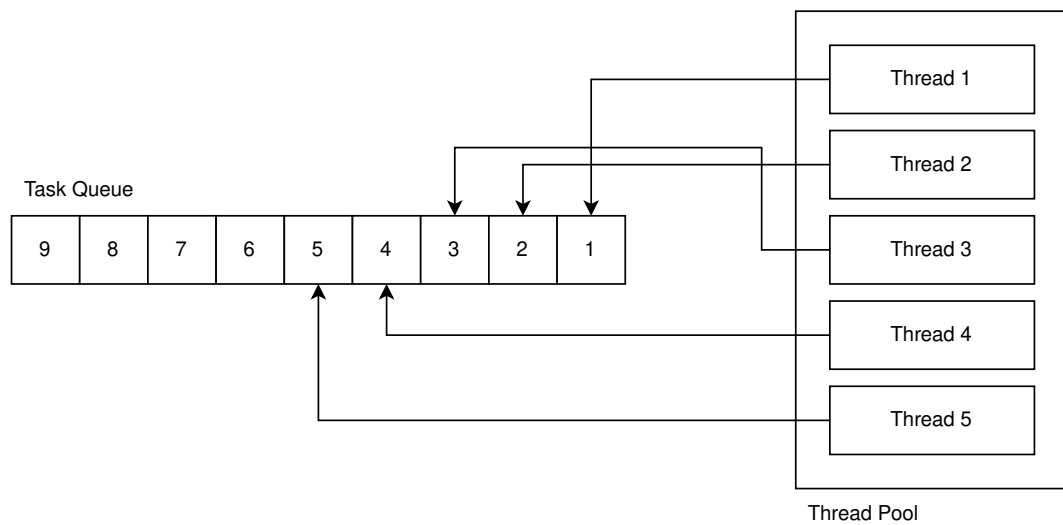


Figura 4.1: Allocazione iniziale dei *Task* per ogni *Thread* della *Pool*. Immagine creata con l'ausilio dell'editor Draw.io[10].

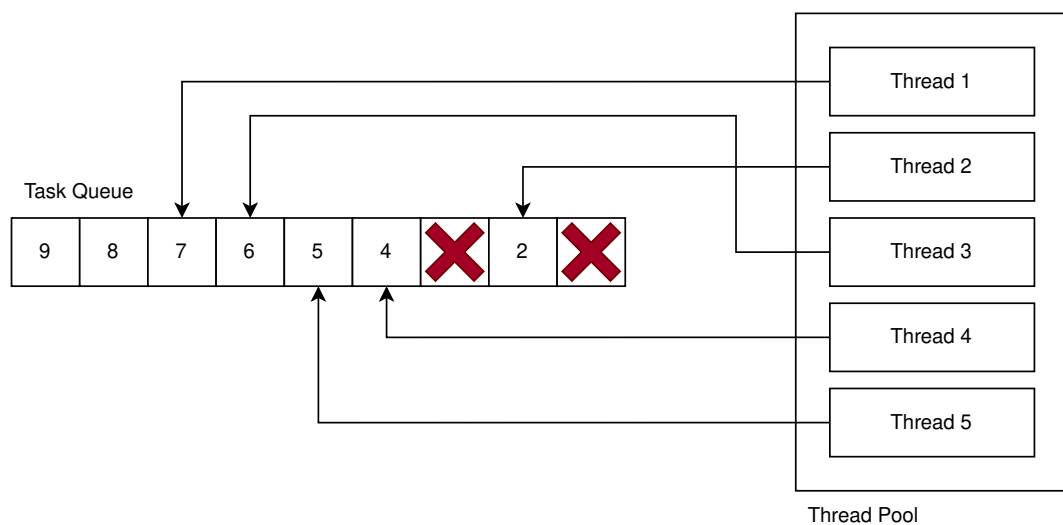


Figura 4.2: Possibile allocazione dei *Task* dopo la terminazione dell'esecuzione del *Task* numero 3 e del *Task* numero 1. Immagine creata con l'ausilio dell'editor Draw.io[10].

4.2 Implementazione di *Thread Pools*

L'implementazione di una *Thread Pool* avviene generalmente tramite l'utilizzo dell'Interfaccia `ExecutorService`: una sotto-Interfaccia di `Executor` che produce oggetti di tipo `Future` per tenere traccia dell'avanzamento dei *Task*. L'interfaccia

dispone di un Metodo per la creazione di una *Thread Pool* contenente un numero fissato di *Thread* e viene implementata in particolare da due Classi che realizzano *Thread Pool* più complesse.

4.2.1 Il Metodo `newFixedThreadPool`

Il Metodo `newFixedThreadPool(int numOfThreads)` permette di creare una *Thread Pool* con un numero fissato di *Worker Thread* atti all'esecuzione dei *Task* loro assegnati. In ogni momento al massimo `numOfThreads` *Thread* saranno attivi e in esecuzione sulla *Pool*. In caso di errori durante l'esecuzione di un *Task*, il *Thread* assegnatogli ne sceglierà un altro.

4.2.2 La Classe `ThreadPoolExecutor`

La Classe `ThreadPoolExecutor` dispone di maggiori parametri per adattarsi all'utilizzo in condizioni differenti: è possibile infatti scegliere il numero minimo e massimo di *Thread* che possono essere allocati in ogni momento a seconda del carico.

La scelta di un corretto numero di *Thread* diviene ora di fondamentale importanza al fine di non sprecare risorse o di allocarne troppe poche.

4.2.3 La Classe `ScheduledThreadPoolExecutor`

Quest'ultima è una reimplementazione della Classe precedente con la differenza che mentre prima i *Task* venivano eseguiti non appena possibile, ora è possibile schedulare l'inizio dell'esecuzione dei singoli *Task*.

4.2.4 Metodi di `ExecutorService`

`ExecutorService` dispone di Metodi per la gestione dell'esecuzione dei vari *Task* ad esso assegnati. Se ne riportano i più importanti.

`execute()`

Metodo ereditato dall'Interfaccia `Executor`, permette di avviare l'esecuzione di un Oggetto di tipo `Runnable` non appena possibile.

`submit()`

Permette di inserire nella Coda un nuovo *Task*: esso può essere sia di tipo `Runnable` - quindi senza Valore di Ritorno - che `Callable`, e quindi con un Valore di Ritorno.

shutdown()

Attende la terminazione di tutti i *Task* e termina i *Thread* della *Pool* per poi distruggerla, non accettando nuovi inserimenti.

shutdownNow()

Tenta di terminare i *Task* in esecuzione, cancella l'esecuzione dei prossimi in coda e non accetta nuovi inserimenti su di essa, ritornando una lista di *Task* che attendevano di eseguire al momento dell'invocazione.

invokeAny()

Esegue i *Task* indicati dalla lista passata come parametro al Metodo stesso, ritornando il risultato del primo *Thread* che esegue correttamente il *Task*.

invokeAll()

Esegue i *Task* indicati dalla lista passata come parametro al Metodo stesso, ritornando una lista di Oggetti di tipo **Future** che contengano lo stato e il risultato dell'esecuzione una volta che tutti hanno terminato.

awaitTermination()

Blocca l'esecuzione del *Thread* principale fin tanto che l'esecuzione dei *Task* non termina.

Capitolo 5

Conclusioni

Nel corso della Storia dell'Informatica, l'avanzamento tecnico e tecnologico verificatosi nell'ambito delle Architetture dei Calcolatori e dei Sistemi Operativi ha permesso grandi passi in avanti nelle dinamiche di utilizzo degli Elaboratori Elettronici. Si è assistito al passaggio dall'esecuzione sequenziale delle operazioni ad un'esecuzione parallela delle stesse, il tutto all'interno di un singolo Elaboratore.

L'opportunità di avere esperienze utente multiprogrammate è stata resa possibile, oltre che dagli Elaboratori non sequenziali, anche da Linguaggi di Programmazione anch'essi non sequenziali che permettano l'esecuzione concorrente di più processi. Tale flessibilità ha permesso un grado di utilizzo delle risorse nettamente superiore al passato, garantendo prestazioni superiori a parità di risorse.

È chiaro come il Linguaggio di Programmazione Java sia stato nel tempo un'ottima piattaforma per lo sviluppo di Software che sfrutti appieno queste possibilità, contribuendo significativamente allo sviluppo dell'Informatica come la conosciamo oggi.

Bibliografia

- [1] Wikipedia. Bubble Sort, Giugno 2023. Articolo disponibile all'url: https://it.wikipedia.org/wiki/Bubble_sort. Pagina visitata il 31 dicembre 2023.
- [2] Wikipedia. Radix Sort, Maggio 2023. Articolo disponibile all'url: https://it.wikipedia.org/wiki/Radix_sort. Pagina visitata il 31 dicembre 2023.
- [3] Wikipedia. Algoritmo di Dijkstra, Aprile 2023. Articolo disponibile all'url: https://it.wikipedia.org/wiki/Algoritmo_di_Dijkstra. Pagina visitata il 31 dicembre 2023.
- [4] Wikipedia. Algoritmo di Bellman-Ford, Maggio 2023. Articolo disponibile all'url: https://it.wikipedia.org/wiki/Algoritmo_di_Bellman-Ford. Pagina visitata il 31 dicembre 2023.
- [5] PlantUML, Graphviz, Ace Editor, Johan Sundström, Steven Nichols e Arwen Vaughan. PlantText UML Editor. Editor disponibile all'url: <https://www.planttext.com/>. Pagina visitata il 31 dicembre 2023.
- [6] Wikipedia. Semaforo, Agosto 2023. Articolo disponibile all'url: [https://it.wikipedia.org/wiki/Semaforo_\(informatica\)](https://it.wikipedia.org/wiki/Semaforo_(informatica)). Pagina visitata il 31 dicembre 2023.
- [7] Wikipedia. Monitor, Marzo 2022. Articolo disponibile all'url: [https://it.wikipedia.org/wiki/Monitor_\(sincronizzazione\)](https://it.wikipedia.org/wiki/Monitor_(sincronizzazione)). Pagina visitata il 31 dicembre 2023.
- [8] Prof. Letizia Leonardi. Costrutti di Sincronizzazione, Ottobre 2022. Dispensa disponibile all'url: <https://www.didattica.agentgroup.unimo.it/didattica/ProgettazioneS0/Lucidi/CostruttiDISincronizzazione-3bis.pdf>. Documento Protetto da Password. Pagina visitata il 31 dicembre 2023.
- [9] Prof. Letizia Leonardi, PhD Mariachiara Puviani. Programmazione Concorrente in Java, Ottobre 2014. Dispensa disponibile all'url: <https://didattica.agentgroup.unimore.it/wiki/images/5/5e/SeminarioJava2014.pdf>. Pagina visitata il 31 dicembre 2023.
- [10] JGraph Ltd. draw.io. Editor disponibile all'url: <https://www.drawio.com/>. Pagina visitata il 31 dicembre 2023.